Deep Learning Practical 1 (DL1.ipynb) — Complete Viva & Code Guide

Zero-Prerequisite, Line-by-Line Explanation & Oral Exam Cheat Sheet for External Viva

1. Practical Aim & 30-Second Examiner Summary

Official Practical Aim (Cell 1): *"To Study of Deep learning Packages: Tensorflow, Keras, Theano and PyTorch. Document the distinct features and functionality of the packages."*

What to say first when the external examiner asks "What is this practical about?":

> *"In this practical, we set up and explore the core Python libraries used for Deep Learning: NumPy, TensorFlow, Keras, and PyTorch. We verify their installations, load the standard MNIST handwritten digits dataset using Keras to inspect its 3D array structure, check GPU hardware acceleration in PyTorch, and compare the architectures of TensorFlow, Keras, Theano, and PyTorch."*

---

2. Complete Cell-by-Cell & Line-by-Line Code Breakdown

Every block below matches the exact order in your Jupyter Notebook (`DL1.ipynb`), showing the exact code, its output, what each word means, and how to explain it aloud.

Block 1 — Cell [13]: Importing NumPy

```
import numpy as np
```

- **Output:** None (loads silently into computer memory).
- **Word-by-Word Breakdown:**
  - `import`: A Python keyword that loads an external library into your program.
  - `numpy`: Short for **Numerical Python**. Computers cannot directly understand images or text; NumPy converts data into multi-dimensional tables of numbers (called arrays or matrices).
  - `as np`: Creates a short nickname (`np`) so we can type `np.array()` instead of writing `numpy.array()` every time.
- **What to tell the examiner:** *"This line imports the NumPy library with the alias `np`. We use it to store and process numerical arrays and matrices, which form the mathematical foundation for all deep learning datasets."*

Block 2 — Cell [14]: Installing & Importing TensorFlow

```
!pip install tensorflow
import tensorflow as tf #Tensorflow
```

- **Output:** `Requirement already satisfied: tensorflow ... (2.14.0)` along with a list of supporting packages (such as `keras`, `tensorboard`, and `numpy`).
- **Line 1 (`!pip install tensorflow`):**
  - `!` (Exclamation mark): Tells Jupyter Notebook to run this line as a system terminal command rather than normal Python code.

- `pip install`: Calls **pip** (Python's package installer) to download and install a library from the internet. If it is already installed, it prints `Requirement already satisfied`.
- `tensorflow`: An open-source Deep Learning library developed by **Google** used to build and train large-scale neural networks.

- **Line 2 (`import tensorflow as tf #Tensorflow`):**
  - `import tensorflow as tf`: Loads TensorFlow into Python and gives it the short nickname `tf`.
  - `#Tensorflow`: Anything starting with `#` is a **comment**. Python ignores it completely; it is just a note for human readers.

- **What to tell the examiner:** *"First, we use the exclamation mark and `pip` to install Google's TensorFlow library from the notebook cell. Then we import TensorFlow into our program using the standard alias `tf`."*

Block 3 — Cell [15]: Checking the TensorFlow Version

```
print(tf.__version__)
```

- **Output:** `2.14.0`
- **Word-by-Word Breakdown:**
  - `print(...)`: Displays the result on the screen.
  - `tf.__version__`: Checks the built-in `__version__` property (written with two underscores on each side, called "dunder") of the TensorFlow library to confirm which version is installed.

- **What to tell the examiner:** *"This line prints the installed version of TensorFlow—which is 2.14.0—to confirm that the library imported properly without errors."*

Block 4 — Cell [16]: Loading the MNIST Handwritten Digit Dataset with Keras

```
#Keras
from keras import datasets
# Load MNIST datasets from keras
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
```

- **Output:** None (loads 70,000 images into the four variables in memory).
- **Line 1 & 3 (`#Keras` and `# Load MNIST...`):** Comments explaining the code section.
- **Line 2 (`from keras import datasets`):**
  - **Keras** is a user-friendly, high-level deep learning API that runs on top of TensorFlow.
  - Instead of importing all of Keras, `from keras import datasets` imports only the built-in `datasets` collection (which contains practice datasets like MNIST, CIFAR-10, and Fashion-MNIST).

- **Line 4 (`(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()`):**
  - `datasets.mnist.load_data()`: Loads the famous **MNIST dataset**, which contains **70,000 grayscale photos of handwritten numbers from 0 to 9** (each image is 28×28 pixels).
  - `train_images`: **60,000 images** used to teach (train) the neural network.
  - `train_labels`: The **60,000 actual digit answers** (0 to 9) corresponding to each training image.
  - `test_images`: **10,000 separate images** kept aside to test the model after training (like a final exam).

- `test_labels`: The **10,000 actual digit answers** for the test images to check the model's accuracy.

- **What to tell the examiner:** *"Here we import the `datasets` module from Keras and call `mnist.load_data()`. It loads 70,000 grayscale images of handwritten digits (0–9) and automatically splits them into 60,000 training images with labels and 10,000 testing images with labels."*

Block 5 — Cell [17]: Inspecting the Shape of the Test Dataset

```
test_images.shape
```

- **Output:** `(10000, 28, 28)`
- **Word-by-Word Breakdown:**
  - `.shape`: A NumPy attribute that tells us the dimensions (size) of an array.
  - `10000`: There are **10,000 images** in the `test_images` set.
  - First `28`: Each image is **28 pixels tall** (28 rows).
  - Second `28`: Each image is **28 pixels wide** (28 columns).
- **Viva Bonus Tip:** If the examiner asks *"What will `train_images.shape` output?"*, answer: **`(60000, 28, 28)`** because the training set has 60,000 images of the exact same 28×28 pixel size! If they ask *"What will `test_labels.shape` output?"*, answer: **`(10000,)`** because it is a 1D list of 10,000 single-digit answers.

Block 6 — Cell [18]: Installing PyTorch with GPU Support

```
#4. PyTorch
!pip3 install torch torchvision torchaudio --extra-index-url
https://download.pytorch.org/whl/cu115
```

- **Output:** `Requirement already satisfied: torch (2.4.1), torchvision (0.19.1), torchaudio (2.4.1)`.
- **Word-by-Word Breakdown:**
  - `!pip3 install`: Runs the Python 3 package installer in the terminal.
  - `torch`: The core **PyTorch** library (developed by Meta/Facebook) for tensor math and deep learning.
  - `torchvision`: PyTorch's companion package for **Computer Vision** (image datasets, pretrained image models, and image transformations).
  - `torchaudio`: PyTorch's companion package for **Audio processing** and speech recognition.
  - `--extra-index-url https://download.pytorch.org/whl/cu115`: Tells `pip` an extra download URL from PyTorch's website so it can fetch GPU-enabled versions compiled for **NVIDIA CUDA 11.5** (`cu115`).
- **What to tell the examiner:** *"This command installs the core PyTorch library along with `torchvision` for image tasks and `torchaudio` for audio tasks, specifying PyTorch's CUDA wheel repository for GPU support."*

Block 7 — Cell [19]: Upgrading the pip Package Installer

```
pip install --upgrade pip
```

- **Output:** `Requirement already satisfied: pip ... (24.2)`

- **Word-by-Word Breakdown:** Updates Python's package installer (`pip`) itself to the newest version so installing large deep learning libraries does not fail due to an outdated installer. (Modern Jupyter allows running `pip` even without the leading `!`).

Block 8 — Cell [20]: Importing PyTorch and its Neural Network Module

```
import torch
import torch.nn as nn
```

- **Output:** None (loads silently into memory).
- **Word-by-Word Breakdown:**
  - `import torch`: Loads the main PyTorch library into the Python session.
  - `import torch.nn as nn`: `torch.nn` is PyTorch's sub-package for building **N**eural **N**etworks (it contains ready-made layers like Linear, Conv2d, activation functions like ReLU, and loss functions). Using `as nn` gives it a short alias.
- **What to tell the examiner:** *"We import the base `torch` package for tensor operations and import `torch.nn` with the alias `nn`, which provides the building blocks like layers and loss functions for neural networks."*

Block 9 — Cell [21]: Checking the PyTorch Version

```
print(torch.__version__)
```

- **Output:** `2.4.1+cu121`
- **What the Output Means:**
  - `2.4.1`: The installed version of PyTorch.
  - `+cu121`: Indicates that this PyTorch package was compiled with support for **NVIDIA CUDA version 12.1**.

Block 10 — Cell [22]: Checking if an NVIDIA GPU (CUDA) is Available

```
torch.cuda.is_available()
```

- **Output:** `False`
- **Word-by-Word Breakdown:**
  - `cuda`: Stands for **Compute Unified Device Architecture**, developed by NVIDIA. It allows deep learning code to run on a Graphics Processing Unit (GPU) instead of a normal CPU, making training dozens of times faster.
  - `torch.cuda.is_available()`: A function that returns `True` if a compatible NVIDIA GPU is detected and ready to use, or `False` if no GPU is available.
  - **Why is the output `False`?** Even though the PyTorch software supports CUDA (`+cu121`), the physical computer where this notebook was executed either does not have an NVIDIA graphics card or does not have GPU drivers active—so PyTorch will run on the CPU.
- **What to tell the examiner:** *"This function checks whether PyTorch can detect an active NVIDIA CUDA GPU for hardware acceleration. Here it returns `False`, meaning no CUDA GPU is available on this system and operations will execute on the CPU."*

---

3. Deep Learning Packages Comparison (Cell 12 Markdown Table Explained)

Your notebook ends with a comparison table of four major packages. Below is the exact table from your notebook enhanced with plain-English explanations so you can answer any theory question about it:

Feature	TensorFlow	Keras	Theano	PyTorch
Developer / Origin	Google Brain	François Chollet (now built into TensorFlow)	MILA (University of Montreal) — pioneer library	Meta AI (Facebook)
Interface	C++, Python, CUDA	Python	Python	Python (with C++/CUDA backend)
Architecture	Complex static graphs (TF 2.x also supports eager/dynamic execution)	Simple high-level API (wraps TensorFlow/PyTorch/JAX)	Static computation graphs	Dynamic computation graphs ("Define-by-Run")
Performance	High performance (optimized for production & clusters)	Depends on the backend engine (e.g., TensorFlow)	High performance (compiles math expressions to C/CUDA)	Competitive, fast GPU execution
Learning Curve	Steep (more complex syntax for low-level control)	Easy (plug-and-play layers, beginner friendly)	Moderate (low-level symbolic math)	Gentle (feels like standard Python/NumPy)
Community Support	Extensive (huge industry adoption)	Growing / Very popular for beginners & rapid builds	Limited (officially discontinued after 2017)	Rapidly increasing (dominant in AI research)
Best Use Case	Large datasets & enterprise production deployment	Small-to-medium datasets & fast prototyping	Historical academic research & symbolic math	Research, custom architectures & prototyping

4. Top 7 External Viva Oral Questions & Ready-to-Speak Answers

Examiner Question	Simple, Confident Answer to Speak in Viva
1. What is a Tensor?	"A tensor is simply a container for numbers across any number of dimensions. A single number is a 0D tensor (scalar), a list is a 1D tensor (vector), a 2D grid is a matrix, and higher-dimensional grids (like a batch of images) are 3D or 4D tensors."

Examiner Question	Simple, Confident Answer to Speak in Viva
2. What is the difference between a Static Graph (TensorFlow 1/Theano) and a Dynamic Graph (PyTorch)?	"In a **Static Graph** (Define-and-Run), you must define the entire mathematical structure of the neural network first, compile it, and only then feed data into it. In a **Dynamic Graph** (Define-by-Run, used by PyTorch), the graph is built on the fly line-by-line as the code runs, which makes debugging with normal Python print statements much easier."
3. What is the difference between TensorFlow and Keras?	"TensorFlow is the complete, low-level engine that handles heavy tensor math, GPU execution, and deployment. Keras is a high-level wrapper API that sits on top of TensorFlow, letting us build neural networks in just a few simple lines of code."
4. Why do we split the MNIST dataset into `train` and `test` sets?	"We use the `train` set (60,000 images) to teach the model patterns, and we keep the `test` set (10,000 unseen images) hidden during training so we can fairly evaluate if the model actually learned to generalize rather than just memorizing the training data."
5. What is Theano and why is its community support listed as 'Limited'?	"Theano was one of the earliest Python deep learning libraries created at the University of Montreal. It pioneered automatic differentiation and GPU compilation, but active development ended in 2017 after industrial frameworks like TensorFlow and PyTorch overtook it."
6. What is the difference between a NumPy array and a PyTorch/TensorFlow Tensor?	"Both store multi-dimensional arrays of numbers, but NumPy arrays can only run on the CPU and cannot automatically calculate gradients. Tensors in PyTorch and TensorFlow can run on GPUs via CUDA and support automatic differentiation for training neural networks."
7. Why did `torch.cuda.is_available()` return `False` even though `torch.__version__` showed `+cu121`?	"`+cu121` means the PyTorch software package was compiled with CUDA 12.1 support, but `torch.cuda.is_available()` checks the actual hardware. It returned `False` because the computer running the notebook did not have an active NVIDIA GPU available."

Deep Learning Practical 2 (DL2.ipynb) — Complete Viva & Code Guide

Feedforward Neural Network (FNN) with Keras & TensorFlow — Zero-Prerequisite Line-by-Line Explanation

1. Practical Aim & 30-Second Examiner Overview

Official Practical Aim (Cell 1): *"To Implement Feedforward neural networks with Keras and TensorFlow: (a) Import the necessary packages, (b) Load the training and testing data (MNIST/CIFAR10), (c) Define the network architecture using Keras, (d) Train the model using SGD, (e) Evaluate the network, (f) Plot the training loss and accuracy."*

What to say first when the external examiner asks "Explain what you did in Practical 2":

> *"In this practical, we built and trained a 3-layer Feedforward Neural Network using Keras and TensorFlow to recognize handwritten digits (0 to 9) from the MNIST dataset. First, we flattened each 28×28 image into 784 pixels, normalized the pixel values between 0 and 1, and one-hot encoded the labels. Then we built a Sequential network with 128 and 64 Sigmoid hidden neurons and 10 Softmax output neurons, trained it for 10 epochs using the SGD optimizer, evaluated its precision and recall (achieving 78% accuracy), and plotted the training loss and accuracy curves."*

---

2. Complete Cell-by-Cell & Line-by-Line Code Breakdown

Block 1 — Cell [2] (Step a): Installing & Importing Required Libraries

```
#installations
!pip install scikit-learn tensorflow matplotlib numpy
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import matplotlib.pyplot as plt
import numpy as np
```

- **Output:** Prints `Requirement already satisfied...` confirming all 4 packages are installed.
- **Line-by-Line Meaning:**
  - `!pip install scikit-learn tensorflow matplotlib numpy`: Runs a terminal command (`!`) to install the four libraries needed: **scikit-learn** (for data encoding and evaluation reports), **tensorflow** (for building the neural network), **matplotlib** (for plotting graphs), and **numpy** (for number array math).
  - `from sklearn.preprocessing import LabelBinarizer`: Imports a tool that converts single-number digit answers (like `3`) into a 10-number binary list (`[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`), called **One-Hot Encoding**.

- `from sklearn.metrics import classification_report`: Imports a function that generates our final exam scorecard (Precision, Recall, F1-score, and Accuracy for every digit 0–9).
- `from tensorflow.keras.models import Sequential`: Imports the `Sequential` model container, which lets us stack neural network layers one after another in a simple straight line.
- `from tensorflow.keras.layers import Dense`: Imports the `Dense` (Fully Connected) layer, where every neuron connects to every neuron in the previous layer.
- `from tensorflow.keras.optimizers import SGD`: Imports **SGD (Stochastic Gradient Descent)**, the optimization algorithm that adjusts the network's internal weights during training to reduce errors.
- `from tensorflow.keras.datasets import mnist`: Imports the built-in **MNIST** dataset of 70,000 handwritten digit images.
- `from tensorflow.keras import backend as K`: Imports Keras's low-level mathematical engine as `K`.
- `import matplotlib.pyplot as plt`: Imports Matplotlib's plotting tool with the short nickname `plt` to draw our Loss/Accuracy graph at the end.
- `import numpy as np`: Imports NumPy with the nickname `np` for handling number arrays.

Block 2 — Cell [3] (Step b - Part 1): Loading, Flattening & Normalizing the Images

```
#grabbing the mnist dataset
((X_train, Y_train), (X_test, Y_test)) = mnist.load_data()
X_train = X_train.reshape((X_train.shape[0], 28 * 28 * 1))
X_test = X_test.reshape((X_test.shape[0], 28 * 28 * 1))
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
```

- **Output:** None (transforms the image arrays in memory).
- **Line 1 (`((X_train, Y_train), (X_test, Y_test)) = mnist.load_data()`):**
  - Loads the 70,000 grayscale handwritten digit images (each 28×28 pixels).
  - `X_train` gets **60,000 training images** and `Y_train` gets their **60,000 true digit answers** (0–9).
  - `X_test` gets **10,000 test images** and `Y_test` gets their **10,000 true digit answers**.
- **Lines 2 & 3 (`X_train.reshape((X_train.shape[0], 28 * 28 * 1))`):**
  - **Why do we reshape? (Very common Viva question!)** A standard Feedforward Neural Network (`Dense` layer) cannot accept a 2D square grid of pixels (`28 × 28`); it requires a flat 1D row of numbers.
  - `X_train.shape[0]` is `60000` (the number of images), and `28 * 28 * 1 = 784` (total pixels per image, where `1` means 1 grayscale channel). So `X_train` changes shape from `(60000, 28, 28)` to **`(60000, 784)`**, and `X_test` changes from `(10000, 28, 28)` to **`(10000, 784)`**.
- **Lines 4 & 5 (`X_train.astype("float32") / 255.0`):**
  - **Why do we divide by 255.0? (Data Normalization!)** In grayscale images, pixel brightness ranges from `0` (pure black) to `255` (pure white) as whole numbers.
  - `.astype("float32")` converts the integers into decimal numbers, and dividing by `255.0` scales every pixel value into a small range between **`0.0` and `1.0`**. Neural networks train much faster and avoid math overflow when inputs are between 0 and 1.

Block 3 — Cell [4] (Step b - Part 2): One-Hot Encoding the Labels

```
lb = LabelBinarizer()
Y_train = lb.fit_transform(Y_train)
Y_test = lb.transform(Y_test)
```

- **Output:** None (converts `Y_train` and `Y_test` into 10-column binary tables).
- **Line-by-Line Meaning:**
  - `lb = LabelBinarizer()`: Creates the encoder object.
  - `Y_train = lb.fit_transform(Y_train)`: Finds all 10 unique digit classes (0 to 9) in the training answers and converts each single number into a **One-Hot Encoded vector** of 10 numbers. For example, digit `2` becomes `[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]` (a `1` at index 2, and `0` everywhere else).
  - `Y_test = lb.transform(Y_test)`: Applies the exact same 10-number transformation to the test answers.
  - **Why is this needed?** Because our neural network's final layer has 10 output neurons (giving 10 probabilities), so the true answers must also be 10 numbers long to calculate the error!

Block 4 — Cell [5] (Step c): Defining the Neural Network Architecture

```
#building the model
model = Sequential()
model.add(Dense(128, input_shape=(784,), activation="sigmoid"))
model.add(Dense(64, activation="sigmoid"))
model.add(Dense(10, activation="softmax"))
```

- **Output:** None (constructs the 3-layer brain in memory).
- **Line-by-Line Meaning:**
  - `model = Sequential()`: Initializes an empty sequential model where data flows forward from layer to layer (hence **Feedforward Neural Network**).
  - `model.add(Dense(128, input_shape=(784,), activation="sigmoid"))`: Adds the **First Hidden Layer**:
    - `input_shape=(784,)`: Tells the network that each incoming image has **784 numbers** (28×28 pixels).
    - `128`: Creates **128 neurons** in this first hidden layer.
    - `activation="sigmoid"`: Uses the **Sigmoid activation function**, which squashes each neuron's output between `0` and `1` so the network can learn non-linear patterns.
  - `model.add(Dense(64, activation="sigmoid"))`: Adds the **Second Hidden Layer** with **64 neurons** and the `sigmoid` activation function to extract more compact features.
  - `model.add(Dense(10, activation="softmax"))`: Adds the **Output Layer** with **10 neurons** (one for each digit class `0` through `9`).
    - `activation="softmax"`: Converts the 10 raw scores into **10 probabilities that add up to 1.0 (100%)**. For example, if neuron #7 has a probability of `0.92` (92%), the model predicts the digit is `7`!

Block 5 — Cell [6] (Step d): Compiling & Training the Model with SGD

```
sgd = SGD(0.01)
epochs=10
```

```
model.compile(loss="categorical_crossentropy", optimizer=sgd,metrics=["accuracy"])
H = model.fit(X_train, Y_train, validation_data=(X_test, Y_test),epochs=epochs,
batch_size=128)
```

- **Output:** Prints progress from `Epoch 1/10` to `Epoch 10/10` (`469/469` steps per epoch). Training accuracy rises from **14.61%** (Epoch 1) to **76.27%** (Epoch 10), while validation accuracy reaches **77.57%**.
- **Line-by-Line Meaning:**
  - `sgd = SGD(0.01)`: Sets up the **Stochastic Gradient Descent** optimizer with a **learning rate of `0.01`** (the step size used when updating neuron weights).
  - `epochs = 10`: Tells the network to pass through the entire 60,000-image training dataset **10 full times**.
  - `model.compile(...)`: Prepares the model for training with three settings:
    - `loss="categorical_crossentropy"`: The mathematical error function used for multi-class classification (10 classes) with one-hot encoded labels.
    - `optimizer=sgd`: Connects our SGD optimizer to minimize the loss.
    - `metrics=["accuracy"]`: Tracks the percentage of images guessed correctly during training.
  - `H = model.fit(...)`: Starts the actual training loop and stores the epoch-by-epoch history inside variable `H`:
    - `X_train, Y_train`: The training data and answers used to update weights.
    - `validation_data=(X_test, Y_test)`: Tests the model on the 10,000 unseen test images after every epoch (producing `val_loss` and `val_accuracy`).
    - `batch_size=128`: Feeds 128 images at a time before updating weights. *(Viva Tip: 60,000 images ÷ 128 batch size = **469 batches/steps per epoch**, which is why the output shows `469/469`!)*

Block 6 — Cell [7] (Step e): Making Predictions & Printing the Classification Report

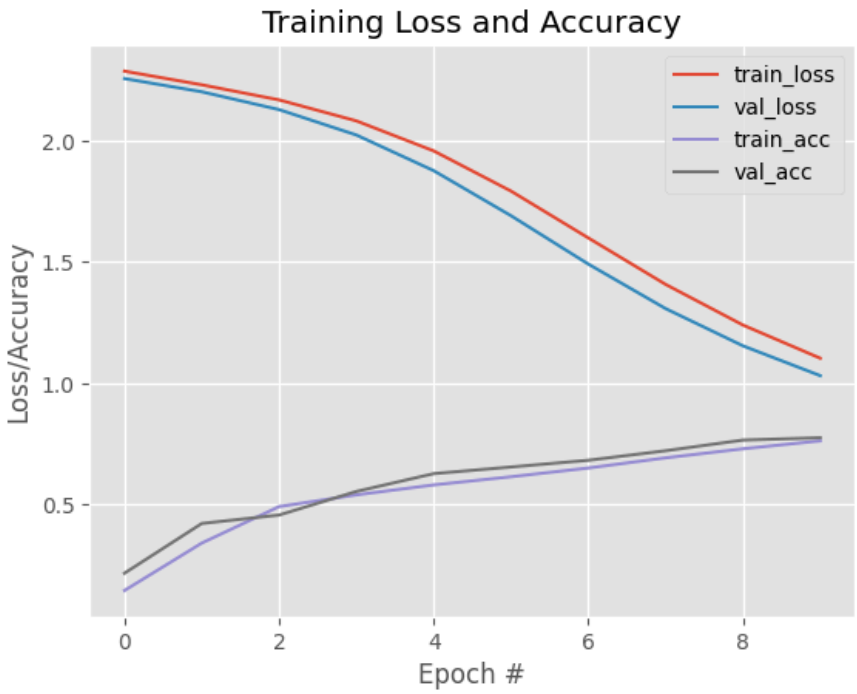
```
#making the predictions
predictions = model.predict(X_test, batch_size=128)
print(classification_report(Y_test.argmax(axis=1),predictions.argmax(axis=1),target_nam
es=[str(x) for x in lb.classes_]))
```

- **Output:** Prints a table of `precision`, `recall`, `f1-score`, and `support` for digits `0`–`9`, with an overall **accuracy of `0.78` (78%)** across `10000` test images.
- **Line-by-Line Meaning:**
  - `predictions = model.predict(X_test, batch_size=128)`: Feeds the 10,000 test images into our trained network (128 at a time). For each image, it outputs 10 probabilities (one for each digit 0–9).
  - `Y_test.argmax(axis=1)` and `predictions.argmax(axis=1)`: Because each row has 10 numbers, `.argmax(axis=1)` picks the **column index (0 to 9) that has the biggest number** in each row—turning the 10 probabilities back into a single predicted digit (like `7`) so we can compare the true digit with the predicted digit!
  - `target_names=[str(x) for x in lb.classes_]`: Converts class numbers `[0, 1, ..., 9]` into text labels `['0', '1', ..., '9']` for the report rows.
  - `print(classification_report(...))`: Prints four key metrics that examiners love to ask about:

- **Precision**: When the model guesses a digit (e.g., `0`), how often is it right? (`0.83` = 83% for digit 0).
- **Recall**: Out of all real images of that digit, how many did the model successfully catch? (`0.97` = 97% for digit 0).
- **F1-Score**: The combined harmonic average of Precision and Recall.
- **Support**: The exact number of test images for each digit (adds up to `10000`).

Block 7 — Cell [8] (Step f): Plotting the Training Loss and Accuracy

```
#plotting the training loss and accuracy
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, epochs), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, epochs), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, epochs), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, epochs), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```



Output Graph from Cell [8]: Downward curves show Loss (error) falling from ~2.28 to ~1.03; upward curves show Accuracy rising from ~0.15 to ~0.78 over 10 epochs.

- **Line-by-Line Meaning:**
  - `plt.style.use("ggplot")`: Applies a clean gray-background grid style to the chart.
  - `plt.figure()`: Opens a new blank chart canvas.
  - `np.arange(0, epochs)`: Creates the X-axis numbers `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]` for the 10 epochs.
  - The four `plt.plot(...)` lines draw the 4 curves stored inside `H.history`: training error (`loss`), test error (`val_loss`), training accuracy (`accuracy`), and test accuracy (`val_accuracy`).

- `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`, `plt.legend()`: Adds the heading, axis labels, and color legend box in the corner.

---

3. Top 7 External Viva Oral Questions & Ready-to-Speak Answers

Examiner Question	Simple, Confident Answer to Speak in Viva
1. Why is it called a Feedforward Neural Network (FNN)?	"Because information flows strictly in one forward direction—from the input layer (784 pixels), through the hidden layers (128 and 64 neurons), to the output layer (10 neurons)—without any loops or cycles."
2. Why did we reshape 28×28 images into 784 and divide by 255.0?	"Dense layers only accept 1D vectors, so we flatten the 28×28 2D grid into 784 pixels. We divide by 255.0 to normalize pixel brightness from 0–255 into the 0.0–1.0 range, which helps gradient descent converge faster."
3. Why did we use `sigmoid` in hidden layers and `softmax` in the output layer?	"`Sigmoid` maps hidden neuron values between 0 and 1 to learn non-linear patterns. `Softmax` is used in the final layer of multi-class problems because it converts the 10 outputs into a probability distribution that sums to 100%."
4. What is SGD and what does the number `0.01` mean?	"SGD stands for Stochastic Gradient Descent. It is the optimizer that updates network weights using backpropagation to minimize loss. `0.01` is the learning rate, which controls how big a step we take toward the minimum error."
5. What is the difference between an Epoch and a Batch Size?	"An **epoch** (10 here) is one complete pass through the entire 60,000 training images. The **batch size** (128 here) is the number of images the model processes at one time before updating its weights once (60,000 ÷ 128 = 469 batches per epoch)."
6. Why did we use `categorical_crossentropy` as the loss function?	"We use `categorical_crossentropy` whenever we have more than two classes (10 digits here) and our target labels (`Y_train`, `Y_test`) are one-hot encoded using `LabelBinarizer`."
7. How could you easily increase the accuracy of this model from 78% to over 95%?	"By replacing the `sigmoid` activation in the hidden layers with `relu` (which avoids the vanishing gradient problem), using the `Adam` optimizer instead of plain SGD, or training for more epochs."

Deep Learning Practical 3 (DL3.ipynb) — Complete Viva & Code Guide

Convolutional Neural Network (CNN) Image Classification on CIFAR-10 — Line-by-Line Explanation

1. Practical Aim & 30-Second Examiner Overview

Official Practical Aim (Cell 1): *"To Build the Image classification model by dividing the model into following 4 stages: (a) Loading and preprocessing the image data, (b) Defining the model's architecture, (c) Training the model, (d) Estimating the model's performance."*

What to say first when the external examiner asks "Explain what you did in Practical 3":

> *"In this practical, we built a Convolutional Neural Network (CNN) using TensorFlow and Keras to classify 32×32 RGB color photos from the CIFAR-10 dataset into 10 object categories like airplanes, cars, birds, and trucks. Unlike Practical 2 where we flattened images right away, here we used 2D Convolution layers (`Conv2D`) to extract visual edges and shapes, `MaxPooling2D` layers to shrink the feature maps, and `Dense` layers at the end to classify the objects—reaching 54.52% test accuracy in just 1 epoch using the Adam optimizer."*

---

2. Complete Cell-by-Cell & Line-by-Line Code Breakdown

Block 1 — Cell [1]: Importing the Required Libraries

```
#importing the libraries
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
```

- **Output:** Prints system info messages stating that no NVIDIA GPU drivers were found, so TensorFlow will run on the CPU.
- **Line-by-Line Meaning:**
  - `import matplotlib.pyplot as plt`: Loads Matplotlib's plotting module with the nickname `plt` so we can display the color photos from the dataset.
  - `import tensorflow as tf`: Loads Google's TensorFlow library with the nickname `tf`.
  - `from tensorflow.keras import datasets, layers, models`: Imports three essential modules from Keras:
    - `datasets`: Gives us access to built-in image datasets (here we load **CIFAR-10**).
    - `layers`: Gives us the building blocks of a CNN (`Conv2D`, `MaxPooling2D`, `Flatten`, and `Dense`).
    - `models`: Gives us `models.Sequential()` to stack our layers in order from input to output.

Block 2 — Cell [2] (Stage a - Part 1): Loading & Normalizing the CIFAR-10 Dataset

```
#grabbing CIFAR10 dataset
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
train_images, test_images = train_images / 255.0, test_images / 255.0
```

- **Output:** `Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz (170498071 bytes)`.
- **Line 1 (`datasets.cifar10.load_data()`):**
  - **What is CIFAR-10? (Very Common Viva Question!)** CIFAR-10 is a standard dataset of **60,000 tiny color (RGB) images** across **10 everyday object classes**.
  - Each image has shape `(32, 32, 3)`: **32 pixels high**, **32 pixels wide**, and **3 color channels** (Red, Green, Blue). *(Compare this to MNIST in Practical 2, which was 28×28×1 grayscale!)*
  - `train_images` & `train_labels`: **50,000 color images** and their integer labels (`0` to `9`) used for training.
  - `test_images` & `test_labels`: **10,000 color images** and their integer labels kept separate for testing.
- **Line 2 (`train_images, test_images = train_images / 255.0, test_images / 255.0`):**
  - **Data Normalization:** Color pixel brightness starts as whole numbers from `0` to `255`. Dividing by `255.0` scales all pixel values into decimals between **`0.0` and `1.0`** so the network trains smoothly without large gradient jumps.
  - **Notice we did NOT flatten the images here!** Why? Because a CNN needs the original 2D spatial height and width (`32×32×3`) so its sliding filters can recognize shapes like wings, wheels, or ears!

Block 3 — Cell [3] (Stage a - Part 2): Displaying Sample Images from the 10 Categories

```
#showing images of mentioned categories
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer','dog', 'frog', 'horse',
'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(10):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



Output from Cell [3]: The first 10 training images (32×32 RGB) plotted with their category names underneath.

- **Line-by-Line Meaning:**
  - `class_names = ['airplane', ..., 'truck']`: Stores the 10 English names in order from index `0` (`airplane`) to index `9` (`truck`), because `train_labels` only contains numbers `0`–`9`.
  - `plt.figure(figsize=(10,10))`: Creates a 10×10 inch blank figure window.
  - `for i in range(10):`: A loop that runs 10 times (for `i = 0, 1, ..., 9`) to plot the first 10 training photos.

- `plt.subplot(5,5,i+1)`: Divides the figure into a 5×5 grid (25 small boxes) and selects box number `i+1` (boxes 1 through 10).
- `plt.xticks([])` and `plt.yticks([])`: Hides the numbered X and Y ruler ticks around each small image.
- `plt.grid(False)`: Ensures no grid lines are drawn over the photos.
- `plt.imshow(train_images[i])`: Displays the `i`-th color photo on the screen.
- `plt.xlabel(class_names[train_labels[i][0]])`: Takes the numeric label of image `i` (for example, `6` for the first image), looks up index `6` in `class_names` (`'frog'`), and prints it under the photo.
- `plt.show()`: Renders the complete grid of images.

Block 4 — Cell [4] (Stage b): Defining the CNN Architecture & Printing `model.summary()`

```
#building CNN model
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))

model.summary()
```

- **How the 3 Core CNN Layer Types Work (Explain this clearly in Viva!):**
  - **1. `Conv2D(filters, (3, 3), activation='relu')` (Convolutional Layer):** Slides small **3×3 pixel magnifying glasses (filters)** across the image to detect visual features like edges, corners, textures, and shapes. `relu` replaces negative numbers with `0` so the network learns non-linear patterns quickly.
  - **2. `MaxPooling2D((2, 2))` (Pooling / Downsampling Layer):** Looks at every **2×2 pixel block** and keeps only the single **largest number**. This cuts the height and width in half, reducing memory size and keeping only the strongest features.
  - **3. `Flatten()` & `Dense(...)` (Classification Layers):** Once the image features are extracted into a compact `(4, 4, 64)` cube, `Flatten()` unrolls it into a flat list of `1,024` numbers (`4 × 4 × 64 = 1024`), and the two `Dense` layers classify those features into the **10 object categories**.

Complete Walkthrough of the `model.summary()` Output Table (Examiners love asking how the shapes change!):

Layer in Code	Output Shape	Param #	Why did the Shape & Parameter Count become this exact number?
1. `Conv2D(32, (3,3), input_shape=(32,32,3))`	`(None, 30, 30, 32)`	`896`	Sliding a `3×3` filter on a `32×32` image without border padding trims 1 pixel off each side (`32 - 3 + 1 = 30`). `32` filters give depth 32. Params: `(3×3×3 weights + 1 bias) × 32 = 896`.

Layer in Code	Output Shape	Param #	Why did the Shape & Parameter Count become this exact number?
2. `MaxPooling2D((2,2))`	`(None, 15, 15, 32)`	`0`	A 2×2 max-pooling window cuts height and width in half (`30 ÷ 2 = 15`). Pooling just picks the max value, so it has **0 weights** to learn.
3. `Conv2D(64, (3,3))`	`(None, 13, 13, 64)`	`18496`	A 3×3 filter trims 2 pixels off 15×15 (`15 - 3 + 1 = 13`) and applies `64` filters. Params: `(3×3×32 + 1) × 64 = 18,496`.
4. `MaxPooling2D((2,2))`	`(None, 6, 6, 64)`	`0`	Divides 13×13 by 2 (`13 ÷ 2 = 6.5 → 6`, dropping the odd border pixel), leaving a 6×6 grid across `64` channels.
5. `Conv2D(64, (3,3))`	`(None, 4, 4, 64)`	`36928`	A 3×3 filter trims 2 pixels off 6×6 (`6 - 3 + 1 = 4`), producing a 4×4×64 feature cube. Params: `(3×3×64 + 1) × 64 = 36,928`.
6. `Flatten()`	`(None, 1024)`	`0`	Multiplies `4 × 4 × 64 = 1024` to unroll the 3D cube into a flat 1D vector of `1,024` numbers (`0` learnable parameters).
7. `Dense(64, activation='relu')`	`(None, 64)`	`65600`	Connects all `1,024` flattened inputs to `64` hidden neurons. Params: `(1024 × 64 weights) + 64 biases = 65,600`.
8. `Dense(10)`	`(None, 10)`	`650`	Output layer with **10 neurons** (one score/logit for each of the 10 CIFAR-10 classes). Params: `(64 × 10) + 10 = 650`. **Total Params = 122,570**.

Block 5 — Cell [5] (Stages c & d): Compiling, Training & Estimating Performance

```
#model compilation
model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),metrics=['accuracy'])
epochs = 1
h = model.fit(train_images, train_labels, epochs=epochs, validation_data=(test_images, test_labels))
```

- **Output:** `1563/1563 [==============================] - 151s 92ms/step - loss: 1.5268 - accuracy: 0.4407 - val_loss: 1.2683 - val_accuracy: 0.5452`
- **Line-by-Line Meaning:**
  - `optimizer='adam'`: Uses the **Adam (Adaptive Moment Estimation)** optimizer, which automatically adjusts the learning rate for each weight and converges much faster than plain SGD.
  - `loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)`:
    - **Why `SparseCategoricalCrossentropy`?** Because our labels (`train_labels`) are raw single integers (`0` to `9`) rather than 10-column one-hot vectors!
    - **Why `from_logits=True`?** Because our final `Dense(10)` layer had no `softmax` activation, so it outputs raw scores (called **logits**). Telling the loss function `from_logits=True` makes TensorFlow compute the Softmax probabilities internally in a more numerically stable way.

- `epochs = 1`: Trains the CNN for 1 pass through the 50,000 training images so execution finishes in ~2.5 minutes during the lab exam.
- `h = model.fit(...)`: Trains the model on `(train_images, train_labels)` and evaluates it on `validation_data=(test_images, test_labels)`.
  - **Why `1563/1563` steps?** Keras's default batch size is **32 images per batch**. Since `50,000 training images ÷ 32 = 1,562.5`, it takes **1,563 batches** to finish 1 epoch!
  - **Performance Achieved:** Training accuracy is **44.07%** and test/validation accuracy (`val_accuracy`) is **54.52%** after just 1 epoch (training for 10 epochs increases this to ~70%+).

---

3. Top 6 External Viva Oral Questions & Ready-to-Speak Answers

Examiner Question	Simple, Confident Answer to Speak in Viva
1. Why did we use a CNN (`Conv2D`) here instead of a simple Feedforward network (`Dense`) like in Practical 2?	"Color photos have 2D spatial structure (`32×32×3`). If we flatten a color photo immediately, we lose the spatial relationship between neighboring pixels. `Conv2D` uses 3×3 filters to preserve spatial structure and detect local patterns like edges, textures, and shapes."
2. What is the purpose of `MaxPooling2D((2, 2))`?	"MaxPooling downsamples the feature maps by taking the maximum value in each 2×2 window. This reduces the height and width by half, lowers the number of parameters to prevent overfitting, and makes the model invariant to small shifts in the object's position."
3. Why did the first `Conv2D` output shape change from `32×32` to `30×30`?	"Because by default, `Conv2D` uses `valid` padding (no zero-padding around the borders). When a 3×3 filter slides across a 32×32 image, its center cannot reach the outermost 1-pixel border on each side, reducing the size by `3 - 1 = 2` pixels (`32 - 2 = 30`)."
4. What does `None` mean in `(None, 30, 30, 32)` inside `model.summary()`?	"`None` represents the dynamic **batch size** dimension. It means the model can accept any number of images at once (for example, 32 images per batch during training, or 1 single image during testing)."
5. What is the difference between `CategoricalCrossentropy` (Practical 2) and `SparseCategoricalCrossentropy` (Practical 3)?	"Both are used for multi-class classification, but `CategoricalCrossentropy` requires the labels to be One-Hot Encoded (like `[0, 0, 1, 0, ...]`), whereas `SparseCategoricalCrossentropy` works directly with single integer labels (`0` to `9`) without needing `LabelBinarizer`."
6. Why is `val_accuracy` (54.52%) higher than training `accuracy` (44.07%) in Epoch 1?	"Because training accuracy is averaged across all 1,563 batches *while* the model is still learning from scratch (starting at 10% on batch 1), whereas `val_accuracy` is measured *at the very end* of Epoch 1 after all 1,563 weight updates have already improved the model."

Deep Learning Practical 4 (DL4.ipynb) — Complete Viva & Code Guide

Autoencoder for ECG Heartbeat Anomaly Detection — Zero-Prerequisite Line-by-Line Explanation

1. Practical Aim & 30-Second Examiner Overview

Official Practical Aim (Cell 1): *"To Use Autoencoder to implement anomaly detection. Build the model by using: (a) Import required libraries, (b) Upload / access the dataset, (c) Encoder converts it into latent representation, (d) Decoder networks convert it back to the original input, (e) Compile the models with Optimizer, Loss, and Evaluation Metrics."*

What to say first when the external examiner asks "Explain what you did in Practical 4":

> *"In this practical, we built an Autoencoder neural network to detect abnormal heartbeats (anomalies) in an ECG dataset of 4,998 recordings. Each heartbeat has 140 voltage readings and a label (`1.0` for normal, `0.0` for abnormal). We trained the Autoencoder ONLY on normal heartbeats: the Encoder compresses the 140 readings down to an 8-number latent bottleneck, and the Decoder reconstructs them back to 140 numbers. When tested on unseen heartbeats, normal heartbeats have tiny reconstruction error, while abnormal heartbeats produce an error above our threshold (`0.00972`), allowing us to detect anomalies with 94.2% accuracy."*

---

2. Complete Cell-by-Cell & Line-by-Line Code Breakdown

Block 1 — Cell [1] (Steps a & b): Importing Libraries & Loading the ECG Dataset

```
#importing libraries and dataset
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import Model, Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.model_selection import train_test_split
from tensorflow.keras.losses import MeanSquaredLogarithmicError

PATH_TO_DATA = 'http://storage.googleapis.com/download.tensorflow.org/data/ecg.csv'
data = pd.read_csv(PATH_TO_DATA, header=None)
data.head()
```

- **Output:** Displays a table preview of the first **5 rows × 141 columns** (columns `0` to `140`).
- **Line-by-Line Meaning:**
  - `import numpy as np` & `import pandas as pd`: NumPy handles math arrays (mean and standard deviation); Pandas loads and manipulates CSV spreadsheet tables (`DataFrame`).
  - `import tensorflow as tf` & `import matplotlib.pyplot as plt`: TensorFlow builds the neural network; Matplotlib plots the loss graph.

- `from sklearn.metrics import accuracy_score`: Calculates the final percentage of heartbeats classified correctly (`0.942` = 94.2%).
- `from tensorflow.keras.optimizers import Adam`: Imports the Adam optimizer to update network weights.
- `from sklearn.preprocessing import MinMaxScaler`: Scales all negative/positive ECG numbers into the range `0` to `1`.
- `from tensorflow.keras import Model, Sequential`: `Model` is the base class we inherit from to create our custom `AutoEncoder`; `Sequential` stacks layers inside the Encoder and Decoder.
- `from tensorflow.keras.layers import Dense, Dropout`: `Dense` creates fully connected neuron layers; `Dropout` randomly turns off a fraction of neurons during training to prevent overfitting.
- `from sklearn.model_selection import train_test_split`: Splits the dataset into 80% training and 20% testing sets.
- `from tensorflow.keras.losses import MeanSquaredLogarithmicError`: Imports the MSLE reconstruction error function.
- `PATH_TO_DATA = 'http://.../ecg.csv'`: Web URL of the **ECG (Electrocardiogram) dataset** hosted by Google TensorFlow.
- `data = pd.read_csv(PATH_TO_DATA, header=None)`: Loads the CSV file into a table named `data`. Setting `header=None` tells Pandas the file has no column names in row 1, so Pandas numbers the columns `0` to `140`.
- `data.head()`: Prints the first 5 rows. **Important for Viva:** Columns `0` to `139` (first 140 columns) are the 140 voltage points of one heartbeat; **Column `140`** is the label (**`1.0` = Normal heartbeat**, **`0.0` = Anomaly/Arrhythmia**).

Block 2 — Cell [2]: Checking the Shape of the Dataset

```
#finding shape of the dataset
data.shape
```

- **Output:** `(4998, 141)`
- **What to tell the examiner:** *"The dataset contains 4,998 rows (individual patient heartbeats) and 141 columns (140 ECG signal readings per heartbeat plus 1 target label column at index 140)."*

Block 3 — Cell [3]: Splitting Data & Filtering ONLY Normal Heartbeats for Training

```
#splitting training and testing dataset
features = data.drop(140, axis=1)
target = data[140]
x_train, x_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, stratify=target
)
train_index = y_train[y_train == 1].index
train_data = x_train.loc[train_index]
```

- **Output:** None (prepares the training and testing variables in memory).
- **Line-by-Line Meaning:**
  - `features = data.drop(140, axis=1)`: Drops column `140` (`axis=1` means column) so `features` holds only the **140 input ECG readings** (columns `0`–`139`).
  - `target = data[140]`: Stores column `140` (the true labels `1.0` or `0.0`) in `target`.

- `train_test_split(features, target, test_size=0.2, stratify=target)`: Splits the 4,998 rows into **80% training** (~3,998 rows) and **20% testing** (`test_size=0.2`, ~1,000 rows). `stratify=target` keeps the exact same ratio of normal-to-abnormal heartbeats in both sets.
- `train_index = y_train[y_train == 1].index` & `train_data = x_train.loc[train_index]`:
  - **Why do we keep ONLY `y_train == 1` (Normal heartbeats) in `train_data`? (Most Important Viva Question!)**
  - In Autoencoder Anomaly Detection, we train the model **exclusively on Normal data (`1.0`)**. That way, the Autoencoder learns how to reconstruct *normal* heartbeats accurately (low error). When an *abnormal* heartbeat (`0.0`) appears during testing, the model fails to reconstruct it properly, producing a high error that flags the anomaly!

Block 4 — Cell [4]: Scaling the Data between 0 and 1 with `MinMaxScaler`

```
#scaling the data using MinMaxScaler
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
x_train_scaled = min_max_scaler.fit_transform(train_data.copy())
x_test_scaled = min_max_scaler.transform(x_test.copy())
```

- **Output:** None.
- **Line-by-Line Meaning:**
  - `min_max_scaler = MinMaxScaler(feature_range=(0, 1))`: Creates a scaler object that maps all ECG voltage readings (which originally go from negative `-4.5` to positive `+1.6`) into the range **`0.0` to `1.0`**.
  - `x_train_scaled = min_max_scaler.fit_transform(train_data.copy())`: Computes the min and max of each column on the normal training set (`fit`) and scales the values between 0 and 1 (`transform`).
  - `x_test_scaled = min_max_scaler.transform(x_test.copy())`: Scales the test set using the training set's min/max values. *(Viva Tip: We also need values in `[0, 1]` because our Decoder's final layer uses `sigmoid` activation, which outputs values in `[0, 1]`, and `msle` loss requires non-negative numbers!)*

Block 5 — Cell [5] (Steps c & d): Defining the `AutoEncoder` Architecture

```
#creating autoencoder subclass by extending Model class from keras
class AutoEncoder(Model):
  def __init__(self, output_units, ldim=8):
    super().__init__()
    self.encoder = Sequential([
      Dense(64, activation='relu'),
      Dropout(0.1),
      Dense(32, activation='relu'),
      Dropout(0.1),
      Dense(16, activation='relu'),
      Dropout(0.1),
      Dense(ldim, activation='relu')
    ])
    self.decoder = Sequential([
      Dense(16, activation='relu'),
      Dropout(0.1),
      Dense(32, activation='relu'),
      Dropout(0.1),
```

```
      Dense(64, activation='relu'),
      Dropout(0.1),
      Dense(output_units, activation='sigmoid')
    ])

  def call(self, inputs):
    encoded = self.encoder(inputs)
    decoded = self.decoder(encoded)
    return decoded
```

- **Output:** None (defines the `AutoEncoder` class).
- **Line-by-Line Meaning:**
  - `class AutoEncoder(Model):` & `def __init__(self, output_units, ldim=8):`: Defines our custom neural network class. Here `output_units` will be `140` (matching the 140 input columns) and `ldim=8` is the size of the compressed **Latent Representation (Bottleneck)**.
  - **Step c — `self.encoder = Sequential([...])` (The Encoder):**
    - Acts like a funnel that compresses the 140-point ECG wave step-by-step: **`140` → `64` → `32` → `16` → `8` neurons (`ldim`)**.
    - This 8-neuron bottleneck forces the network to capture only the core pattern of a normal heartbeat.
    - `Dropout(0.1)`: Randomly switches off **10% of neurons** during training so the network does not simply memorize the data.
  - **Step d — `self.decoder = Sequential([...])` (The Decoder):**
    - Acts like a reverse funnel that reconstructs the original 140-point ECG wave from the 8-number code: **`8` → `16` → `32` → `64` → `140` neurons (`output_units`)**.
    - The final layer uses `activation='sigmoid'` so every reconstructed point is between `0.0` and `1.0`, matching our scaled input!
  - **`def call(self, inputs):`**: Runs the forward pass—first compresses `inputs` using `self.encoder(inputs)`, then reconstructs it using `self.decoder(encoded)` and returns `decoded`.

Block 6 — Cell [6] (Step e): Compiling & Training the Autoencoder

```
#model configuration
model = AutoEncoder(output_units=x_train_scaled.shape[1])
model.compile(loss='msle', metrics=['mse'], optimizer='adam')
epochs = 20

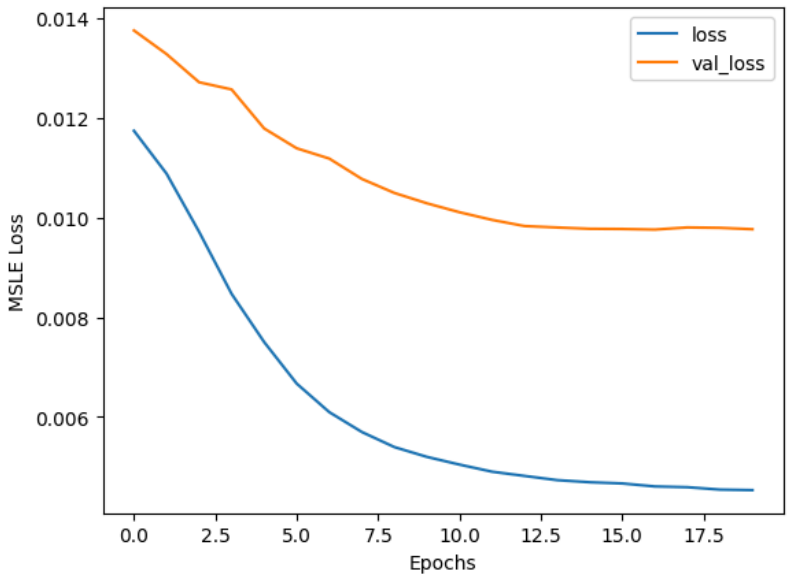
history = model.fit(
    x_train_scaled,
    x_train_scaled,
    epochs=epochs,
    batch_size=512,
    validation_data=(x_test_scaled, x_test_scaled)
)
```

- **Output:** Trains from `Epoch 1/20` to `Epoch 20/20` (`5/5` batches per epoch). Training `loss` drops from `0.0117` down to **`0.0045`**, while `val_loss` settles at **`0.0098`**.
- **Line-by-Line Meaning:**

- `model = AutoEncoder(output_units=x_train_scaled.shape[1])`: Creates the model with `output_units = 140` (the number of columns in `x_train_scaled`).
- `model.compile(loss='msle', metrics=['mse'], optimizer='adam')`: Configures training with **MSLE (Mean Squared Logarithmic Error)** as the loss function, **MSE (Mean Squared Error)** as the monitored metric, and **Adam** as the optimizer.
- `history = model.fit(x_train_scaled, x_train_scaled, ...)`:
  - **Why do we pass `x_train_scaled` TWICE into `model.fit()`? (Guaranteed Viva Question!)**
  - Because in an Autoencoder, the desired target output **IS the input itself**! We want the network to reconstruct `x_train_scaled` as closely as possible, so `X = x_train_scaled` and `Y = x_train_scaled`!

Block 7 — Cell [7]: Plotting the Training and Validation Loss

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel('Epochs')
plt.ylabel('MSLE Loss')
plt.legend(['loss', 'val_loss'])
plt.show()
```



Output Graph from Cell [7]: Blue line shows training error on normal heartbeats dropping to 0.0045; orange line (`val_loss`) stays higher (~0.0098) because the test set contains abnormal heartbeats that have high reconstruction error.

Block 8 — Cell [8]: Finding the Anomaly Threshold & Defining Prediction Function

```
#finding threshold for anomaly and doing predictions
def find_threshold(model, x_train_scaled):
  reconstructions = model.predict(x_train_scaled)
  reconstruction_errors = tf.keras.losses.msle(reconstructions, x_train_scaled)
  threshold = np.mean(reconstruction_errors.numpy()) \
   + np.std(reconstruction_errors.numpy())
  return threshold

def get_predictions(model, x_test_scaled, threshold):
  predictions = model.predict(x_test_scaled)
  errors = tf.keras.losses.msle(predictions, x_test_scaled)
```

```
    anomaly_mask = pd.Series(errors) > threshold
    preds = anomaly_mask.map(lambda x: 0.0 if x == True else 1.0)
    return preds

threshold = find_threshold(model, x_train_scaled)
print(f"Threshold: {threshold}")
```

- **Output:** `Threshold: 0.00972221153034655`
- **How `find_threshold` works:**
  - Reconstructs all normal training heartbeats (`model.predict(x_train_scaled)`) and measures their MSLE error against the original input.
  - Sets the cutoff **`threshold` = Mean Error + 1 Standard Deviation (`np.mean + np.std`) = `0.00972`**. Most normal heartbeats have an error below this number.
- **How `get_predictions` works:**
  - Reconstructs the unseen test heartbeats and computes their MSLE reconstruction `errors`.
  - `anomaly_mask = pd.Series(errors) > threshold`: Checks if the error is **greater than `0.00972`** (`True` = Anomaly!).
  - `anomaly_mask.map(lambda x: 0.0 if x == True else 1.0)`: Converts `True` (Anomaly) to **`0.0`** and `False` (Normal) to **`1.0`** so it matches the dataset's original labels.

Block 9 — Cell [9]: Evaluating Final Anomaly Detection Accuracy

```
#getting accuracy score
predictions = get_predictions(model, x_test_scaled, threshold)
accuracy_score(predictions, y_test)
```

- **Output:** `0.942` (**94.2% Accuracy**)
- **What to tell the examiner:** *"This compares our threshold-based predictions against the true test labels `y_test`, showing that our Autoencoder correctly identifies normal and anomalous heartbeats with 94.2% accuracy."*

---

3. Top 6 External Viva Oral Questions & Ready-to-Speak Answers

Examiner Question	Simple, Confident Answer to Speak in Viva
1. What is an Autoencoder?	"An Autoencoder is an unsupervised neural network trained to copy its input to its output. It has two parts: an **Encoder** that compresses the input into a small latent representation (bottleneck), and a **Decoder** that reconstructs the original data from that compressed code."
2. What is the 'Latent Representation' (Bottleneck) in your code?	"In our code, the latent representation is the final layer of the Encoder with `ldim = 8` neurons. It compresses 140 ECG readings down into just 8 essential numbers, forcing the network to learn the underlying pattern rather than simply copying the input."
3. Why did we train the Autoencoder ONLY on rows where `y_train == 1`?	"Because `1` represents Normal heartbeats. By training only on normal heartbeats, the Autoencoder learns to reconstruct normal patterns with very low error. When it encounters an abnormal heartbeat (`0`), it fails to reconstruct it well, producing a high reconstruction error that reveals the anomaly."

Examiner Question	Simple, Confident Answer to Speak in Viva
4. How did you calculate the Anomaly Threshold (`0.00972`)?	"We calculated the reconstruction error (MSLE) across all normal training samples, and set the threshold equal to the **Mean plus one Standard Deviation** (`np.mean + np.std`) of those errors. Any test heartbeat with an error above `0.00972` is classified as an anomaly (`0.0`)."
5. Why did you use `Dropout(0.1)` between the Dense layers?	"`Dropout(0.1)` randomly deactivates 10% of the neurons during each training step. This prevents the Autoencoder from overfitting (memorizing the training heartbeats) and forces it to learn robust general features."
6. Why did we pass `(x_train_scaled, x_train_scaled)` into `model.fit()` instead of `(x_train, y_train)`?	"Because an Autoencoder is self-supervised—it does not predict class labels during training. Instead, it tries to reconstruct the input signal itself, so the input `x_train_scaled` serves as both the input features and the target output."

Deep Learning Practical 5 (DL5.ipynb) — Complete Viva & Code Guide

Continuous Bag of Words (CBOW) Word2Vec Model — Zero-Prerequisite Line-by-Line Explanation

1. Practical Aim & 30-Second Examiner Overview

Official Practical Aim (Cell 1): *"To Implement the Continuous Bag of Words (CBOW) Model. Stages can be: (a) Data preparation, (b) Generate training data, (c) Train model, (d) Output."*

What to say first when the external examiner asks "Explain what you did in Practical 5":

> *"In this practical, we implemented the Continuous Bag of Words (CBOW) model from Word2Vec to learn numerical word embeddings from a text corpus of 4 sentences (16 unique words). First, we tokenized the words into integer IDs. Second, using a window size of 2, we generated context-target pairs where 4 surrounding context words (2 before and 2 after) are used to predict 1 middle target word. Third, we trained a 3-layer neural network (`Embedding` → `Lambda` mean-averaging → `Dense` Softmax) for 100 epochs to 100% accuracy, and finally extracted the learned 10-dimensional vector embedding for every word."*

---

2. Complete Cell-by-Cell & Line-by-Line Code Breakdown

Block 1 — Cell [1] (Stage a): Importing Libraries & Preparing Text Data

```
# Import necessary libraries
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Lambda, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Sample corpus
corpus = [
    "the cat sat on the mat",
    "the dog sat on the log",
    "cats and dogs are great pets",
    "the mat is soft and warm"
]

# Preprocess text: Tokenization and lowercasing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
total_words = len(tokenizer.word_index) + 1  # +1 for padding

# Convert text to sequences
sequences = tokenizer.texts_to_sequences(corpus)
```

- **Output:** System info messages (loads libraries and builds the vocabulary in memory).

- **Line-by-Line Meaning of Imports:**
  - `import numpy as np`, `import pandas as pd`, `import tensorflow as tf`: Imports NumPy for arrays, Pandas for data structures, and TensorFlow for deep learning.
  - `from tensorflow.keras.models import Sequential`: Stacks our 3 neural network layers in a linear sequence.
  - `from tensorflow.keras.layers import Embedding, Lambda, Dense`:
    - **`Embedding`**: A lookup table layer that converts each word's integer ID into a dense vector of **10 decimal numbers** (word embedding) that captures its semantic meaning.
    - **`Lambda`**: Lets us apply a custom mathematical operation inside the network—here we use it to **average** the 4 context word vectors into 1 combined vector.
    - **`Dense`**: The output classification layer that predicts which word belongs in the center.
  - `from tensorflow.keras.preprocessing.text import Tokenizer`: Splits sentences into lowercase words and assigns a unique number ID (`1, 2, 3...`) to each unique word.
  - `from tensorflow.keras.preprocessing.sequence import pad_sequences`: Ensures all input word sequences have the exact same length (`maxlen=4`).
- **Line-by-Line Meaning of Data Preparation:**
  - `corpus = [...]`: A list of 4 English sentences (in NLP, a collection of text sentences is called a **corpus**). Across these 4 sentences, there are **16 unique words**.
  - `tokenizer = Tokenizer()` & `tokenizer.fit_on_texts(corpus)`: Scans all 4 sentences, converts words to lowercase, and builds a dictionary called `tokenizer.word_index` that maps each of the 16 unique words to an integer ID ordered by frequency (`'the': 1, 'sat': 2, 'on': 3, 'mat': 4, 'and': 5, 'cat': 6, ...`).
  - `total_words = len(tokenizer.word_index) + 1`: Since `len(tokenizer.word_index)` is `16` (IDs 1 to 16), we add `+ 1` to reserve index `0` for padding, making **`total_words = 17`**.
  - `sequences = tokenizer.texts_to_sequences(corpus)`: Converts each text sentence into a list of integer IDs! For example, Sentence 1 (`"the cat sat on the mat"`) becomes **`[1, 6, 2, 3, 1, 4]`**.

Block 2 — Cell [2] (Stage b): Generating CBOW Training Data (Context → Target)

```
## Stage b: Generate Training Data
def generate_training_data(sequences, window_size=2):
    contexts = []
    targets = []

    for sequence in sequences:
        for i in range(window_size, len(sequence) - window_size):
            context = sequence[i - window_size:i] + sequence[i + 1:i + window_size + 1]
            target = sequence[i]
            contexts.append(context)
            targets.append(target)

    return np.array(contexts), np.array(targets)

X, y = generate_training_data(sequences)

# Pad sequences for consistent input shape
X = pad_sequences(X, maxlen=4)  # Adjust maxlen based on context size
```

- **Output:** None (creates input matrix `X` of shape `(8, 4)` and target array `y` of shape `(8,)`).
- **How the Sliding Window Works (Explain this example to the Examiner!):**
  - In CBOW with `window_size=2`, for any middle word at position `i` (the **`target`**), we grab the **2 words to its left** (`sequence[i - 2 : i]`) and the **2 words to its right** (`sequence[i + 1 : i + 3]`) and join them together (`+`) to form a 4-word **`context`**!
  - **Concrete Example from Sentence 1 (`"the cat sat on the mat"` → `[1, 6, 2, 3, 1, 4]`):**
    - When `i = 2`, the middle **Target word** is `"sat"` (ID `2`). Its 4 surrounding **Context words** are `["the", "cat"]` + `["on", "the"]` (IDs `[1, 6, 3, 1]`).
    - When `i = 3`, the middle **Target word** is `"on"` (ID `3`). Its 4 surrounding **Context words** are `["cat", "sat"]` + `["the", "mat"]` (IDs `[6, 2, 1, 4]`).
  - Since each of the 4 sentences has 6 words, each sentence produces 2 training pairs, giving **8 training pairs total** (`X` has 8 rows of 4 context words; `y` has 8 target words).
  - `X = pad_sequences(X, maxlen=4)`: Guarantees every context row in `X` has length 4.

Block 3 — Cell [8] (Stage c): Defining, Compiling & Training the CBOW Model

```
## Stage c: Train Model
# Define CBOW model architecture
model = Sequential()
model.add(Embedding(input_dim=total_words, output_dim=10, input_length=4))
model.add(Lambda(lambda x: tf.reduce_mean(x, axis=1)))  # Average embeddings
model.add(Dense(total_words, activation='softmax'))

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=
['accuracy'])

# Train the model
model.fit(X, y, epochs=100)
```

- **Output:** Trains from `Epoch 1/100` to `Epoch 100/100`. Accuracy starts at `0.1250` (12.5%) in Epoch 1 and reaches **`1.0000`** **(100% accuracy)** from Epoch 67 onward!
- **Line-by-Line Architecture Breakdown:**
  - **Layer 1 — `Embedding(input_dim=total_words, output_dim=10, input_length=4)`:**
    - `input_dim=total_words` (`17`): Total rows in the embedding lookup table (16 words + 1 padding index).
    - `output_dim=10`: Represents each word as a vector of **10 trainable decimal numbers**.
    - `input_length=4`: Takes the **4 context word IDs** as input and outputs 4 vectors of length 10 (shape `4 × 10`).
  - **Layer 2 — `Lambda(lambda x: tf.reduce_mean(x, axis=1))`:**
    - **Why do we use `tf.reduce_mean(x, axis=1)`? (Key Viva Question!)** It computes the element-wise **average** of the 4 context word vectors along `axis=1`, combining the `(4, 10)` matrix into a single **10-number context vector**!
  - **Layer 3 — `Dense(total_words, activation='softmax')`:**
    - Output layer with `17` neurons (`total_words`). `softmax` converts the scores into probabilities summing to 100% so the model can predict the exact middle target word!

- `model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])`: Uses `sparse_categorical_crossentropy` because target `y` contains integer word IDs (not one-hot vectors), and optimizes weights using `adam`.
- `model.fit(X, y, epochs=100)`: Trains the model on the 8 context-target examples for 100 epochs.

Block 4 — Cell [16] (Stage d): Extracting & Displaying the Learned Word Embeddings

```
## Stage d: Output
# Get word embeddings from the trained model
word_embeddings = model.layers[0].get_weights()[0]

# Create a mapping of words to their embeddings
word_index = tokenizer.word_index

print('Vocabulary Size:', len(word_index))
print('Vocabulary Sample:', list(word_index.items())[:10],"\n\n")

embeddings_dict = {word: word_embeddings[idx] for word, idx in word_index.items()}

# Output the embeddings for each word in a structured format
print("{:<10} | {}".format("Word", "Embedding"))
print("-" * 40)
for word, embedding in embeddings_dict.items():
    print("{:<10} | {}".format(word, np.round(embedding, 3)))
```

- **Output:** Prints `Vocabulary Size: 16`, the top 10 word-ID pairs, and a table showing the 10-number embedding vector for all 16 words (e.g., `cat | [ 0.131 -0.106 0.151 ...]` and `dog | [ 0.145 -0.159 0.165 ...]`).
- **Line-by-Line Meaning:**
  - `word_embeddings = model.layers[0].get_weights()[0]`: Pulls the trained weight matrix from the first layer (`model.layers[0]`, which is the `Embedding` layer). Each row `idx` is the learned 10-number vector for word ID `idx`!
  - `word_index = tokenizer.word_index`: Retrieves the dictionary of `{word: id}` pairs.
  - `embeddings_dict = {word: word_embeddings[idx] for word, idx in word_index.items()}`: Creates a dictionary matching each English word string to its 10-number vector from `word_embeddings`.
  - `for word, embedding in embeddings_dict.items(): ... np.round(embedding, 3)`: Loops through all 16 words, rounds each 10-number vector to 3 decimal places, and prints it in a clean table.
  - **Viva Insight on the Output:** Notice that `"cat"` and `"dog"` have nearly identical positive/negative signs across all 10 numbers because they appear in the exact same context (`"the ___ sat on the"`)!

---

3. Top 6 External Viva Oral Questions & Ready-to-Speak Answers

Examiner Question	Simple, Confident Answer to Speak in Viva
1. What is CBOW (Continuous Bag of Words)?	"CBOW is one of the two Word2Vec architectures used to learn word embeddings. It works by taking the surrounding **context words** within a sliding window (for example, 2 words before and 2 words after) and training a neural network to predict the single missing **target word** in the middle."
2. What is the difference between CBOW and Skip-Gram?	"They are exact opposites! **CBOW** takes multiple surrounding context words as input to predict **one center target word** (`Context → Target`). **Skip-Gram** takes **one center word** as input and tries to predict its surrounding **context words** (`Target → Context`). CBOW is much faster to train."
3. Why is it called a 'Bag of Words' if it is a neural network?	"Because in the `Lambda` layer, we average the 4 context word vectors together (`tf.reduce_mean`). When you average vectors, the exact order of those 4 context words is ignored—just like throwing words into a bag—while still preserving their combined meaning."
4. What is a Word Embedding and why is it better than One-Hot Encoding?	"One-hot encoding creates huge, sparse vectors of 0s and 1s where every word is completely independent and similar words share no mathematical relationship. A **Word Embedding** represents each word as a compact, dense vector of decimals (10 numbers here) where words with similar meanings or contexts (like 'cat' and 'dog') have similar vectors."
5. Why did we write `total_words = len(tokenizer.word_index) + 1`?	"Keras's `Tokenizer` assigns word IDs starting from `1` up to `16` (for our 16 unique words), and reserves index `0` for sequence padding (`pad_sequences`). Therefore, the `Embedding` table needs `16 + 1 = 17` rows so index `16` does not go out of bounds."
6. How did we extract the final word embeddings after training?	"The word embeddings are simply the learned weights of the first layer (`Embedding` layer). We extracted them using `model.layers[0].get_weights()[0]` and looked up each word's row index from `tokenizer.word_index`."

Deep Learning Practical 6 (DL6.ipynb) — Complete Viva & Code Guide

Transfer Learning & Fine-Tuning with Pre-Trained MobileNetV2 CNN — Line-by-Line Explanation

1. Practical Aim & 30-Second Examiner Overview

Official Syllabus Aim (Practical 6): *"Object detection / classification using Transfer Learning of CNN architectures: (a) Load in a pre-trained CNN model trained on a large dataset, (b) Freeze parameters (weights) in model's lower convolutional layers, (c) Add custom classifier with several layers of trainable parameters to model, (d) Train classifier layers on training data available for task, (e) Fine-tune hyper-parameters and unfreeze more layers as needed."*

What to say first when the external examiner asks "Explain what you did in Practical 6":

> *"In this practical, we implemented Transfer Learning and Fine-Tuning using Google's pre-trained MobileNetV2 Convolutional Neural Network. Instead of training a deep CNN from scratch, we loaded MobileNetV2 pre-trained on ImageNet without its top 1,000-class layer, froze all its convolutional weights (`layer.trainable = False`), and added our own custom 10-class classifier head (`GlobalAveragePooling2D` + `Dense(256)` + `Dense(10)`). First, we trained only our custom classifier head for 10 epochs. Then, for fine-tuning, we unfroze the last 20 layers of MobileNetV2 and trained for 5 more epochs using a very small learning rate (`1e-5`) to gently adapt the deeper features."*

---

2. Complete Cell-by-Cell & Line-by-Line Code Breakdown

Block 1 — Cell [2]: Importing the Required Libraries

```
# Import necessary libraries
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
import matplotlib.pyplot as plt
```

- **Output:** None (loads packages into memory).
- **Line-by-Line Meaning:**
  - `import numpy as np` & `import tensorflow as tf`: Imports NumPy for creating image arrays and TensorFlow for deep learning.
  - `from tensorflow.keras.applications import MobileNetV2`: Imports **MobileNetV2**, a lightweight, high-speed deep CNN architecture created by Google that comes pre-trained on 1.4 million real-world photos from the **ImageNet** dataset.
  - `from tensorflow.keras.models import Sequential`: Lets us stack the pre-trained `base_model` and our new custom classification layers together in a sequence.
  - `from tensorflow.keras.layers import Dense, GlobalAveragePooling2D`:
    - **`GlobalAveragePooling2D`**: A pooling layer that takes each 2D feature map (`7×7` pixels) coming out of MobileNetV2 and computes its single average number—shrinking shape `(7, 7, 1280)`

into a flat list of `1,280` numbers with **zero extra trainable weights**!
    - **`Dense`**: Fully connected neuron layers used to build our custom classifier at the end.
- `import matplotlib.pyplot as plt`: Imports Matplotlib to plot the accuracy graphs.

Block 2 — Cell [3]: Generating Example Training Images & Labels

```
# Generate example data (Replace this with your actual dataset)
num_samples = 1000  # Number of samples
num_classes = 10    # Number of classes
X_train = np.random.rand(num_samples, 224, 224, 3).astype(np.float32)
y_train = np.random.randint(0, num_classes, num_samples)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
```

- **Output:** `X_train shape: (1000, 224, 224, 3)` and `y_train shape: (1000,)`.
- **Line-by-Line Meaning:**
    - `num_samples = 1000` & `num_classes = 10`: Creates a dataset of **1,000 sample images** across **10 target classes** (`0` to `9`).
    - `X_train = np.random.rand(num_samples, 224, 224, 3).astype(np.float32)`: Generates 1,000 synthetic RGB color images where each image is **224 pixels high, 224 pixels wide, with 3 color channels** (Red, Green, Blue) and pixel values between `0.0` and `1.0`.
        - **Why `224, 224, 3`? (Common Viva Question!)** Because `224×224×3` is the standard input image size that ImageNet architectures (like MobileNetV2, VGG16, and ResNet50) are designed to accept!
    - `y_train = np.random.randint(0, num_classes, num_samples)`: Generates 1,000 random integer labels between `0` and `9`.

Block 3 — Cell [4] (Stage a): Loading the Pre-Trained `MobileNetV2` Base Model

```
# Load pre-trained MobileNetV2 model + higher-level layers
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Display the model architecture
base_model.summary()
```

- **Output:** Prints the 154 internal layers of `mobilenetv2_1.00_224` (Depthwise Separable Convolutions, BatchNormalization, ReLU, and Residual Add blocks), ending with output shape **`(None, 7, 7, 1280)`** and **`Total params: 2,257,984`**.
- **Explain the 3 Parameters of `MobileNetV2(...)` (Guaranteed Viva Question!):**
    - **1. `weights='imagenet'`:** Loads the pre-trained weights learned from the **ImageNet dataset** (1.4 million images, 1,000 classes). This gives our model instant knowledge of how to detect edges, shapes, colors, and textures!
    - **2. `include_top=False`: Removes the original final 1,000-neuron classification layer** ("the top") of MobileNetV2. Why? Because our task has only `10` classes, so we want to keep MobileNetV2's convolutional feature extractor and put our *own* 10-class classifier on top!
    - **3. `input_shape=(224, 224, 3)`:** Sets the input image dimensions to 224×224 RGB. As the image passes through MobileNetV2, its spatial size shrinks from `224×224×3` down to **`7×7×1280`** (1,280 feature maps of size 7×7).

Block 4 — Cell [5] (Stage b): Freezing the Lower Convolutional Layers

```
# Freeze the lower convolutional layers
for layer in base_model.layers:
    layer.trainable = False

# Check which layers are frozen
frozen_layers = [layer.name for layer in base_model.layers if not layer.trainable]
print("Frozen layers:", frozen_layers)
```

- **Output:** Prints `Frozen layers: ['input_1', 'Conv1', 'bn_Conv1', ..., 'out_relu']` (all 154 layers of `base_model`).
- **Line-by-Line Meaning:**
  - `for layer in base_model.layers: layer.trainable = False`: Loops through every layer in `base_model` and **locks (freezes) its weights** so backpropagation will not change them during training.
  - **Why do we freeze the base model layers?**
    1. It preserves the valuable visual features already learned from 1.4 million ImageNet photos so random initial gradients from our new Dense layers don't destroy them.
    2. It avoids training 2.25 million parameters, making training much faster and preventing overfitting on small datasets!
  - `frozen_layers = [layer.name for layer in base_model.layers if not layer.trainable]`: List comprehension that checks every layer where `layer.trainable` is `False` and prints its name to confirm all layers are frozen.

Block 5 — Cell [6] (Stage c): Adding the Custom Classifier Head & Compiling

```
# Create a new model with the base and custom classifier
model = Sequential()
model.add(base_model)
model.add(GlobalAveragePooling2D())
model.add(Dense(256, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))  # Adjust the number of classes as
needed

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
['accuracy'])

# Display the complete model architecture
model.summary()
```

Complete Walkthrough of the `model.summary()` Output Table:

Layer in `model.summary()`	Output Shape	Param #	What does this layer do & how are Parameters calculated?
1. `mobilenetv2_1.00_224`	`(None, 7, 7, 1280)`	`2257984`	The frozen pre-trained base model. Takes a `224×224×3` image and extracts `1,280` feature maps of size `7×7`. All **2,257,984 parameters are Non-trainable (frozen)**.

Layer in `model.summary()`	Output Shape	Param #	What does this layer do & how are Parameters calculated?
2. **`GlobalAveragePooling2D()`**	`(None, 1280)`	`0`	Takes the average of all 49 pixels (7×7) in each of the `1,280` feature maps, turning `(7, 7, 1280)` into a flat 1D vector of **`1,280` numbers** (`0` weights).
3. **`Dense(256, activation='relu')`**	`(None, 256)`	`327936`	Our custom hidden classification layer with 256 ReLU neurons. Params: `(1280 inputs × 256 weights) + 256 biases = 327,936` (**Trainable**).
4. **`Dense(10, activation='softmax')`**	`(None, 10)`	`2570`	Our custom output layer with 10 Softmax neurons (probabilities for our 10 classes). Params: `(256 × 10) + 10 = 2,570` (**Trainable**).

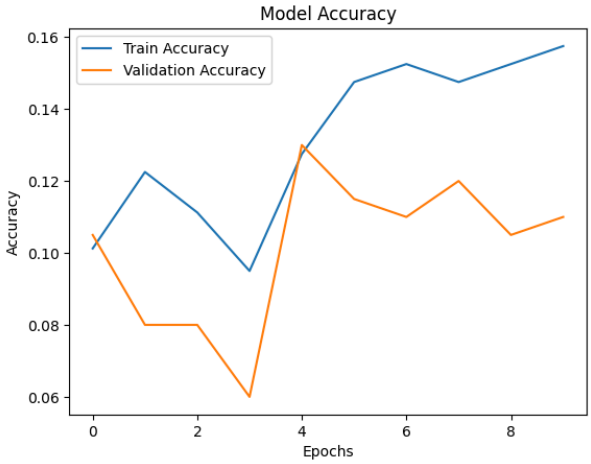
- **Viva Highlight on Parameter Counts:** Notice at the bottom of `model.summary()`:
  - `Total params: 2,588,490`
  - **`Trainable params: 330,506`** (Only our 2 new `Dense` layers: `327,936 + 2,570 = 330,506`!)
  - **`Non-trainable params: 2,257,984`** (All 154 frozen layers inside `MobileNetV2`!)

Block 6 — Cell [7] (Stage d): Training the Classifier Layers & Plotting Accuracy

```
# Train the classifier layers
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

# Plot training history (optional)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

- **Line-by-Line Meaning:**
  - `validation_split=0.2`: Automatically uses **80% of `X_train` (800 images) for training** and holds back **20% (200 images) for validation**.
  - **Why does the output show `25/25` steps per epoch?** Because `800 training images ÷ 32 batch_size = 25 batches`!
  - **Why is accuracy around 10%–16%? (Important Viva Tip!)** Because in Cell [3] we generated **synthetic random noise (`np.random.rand`)** as dummy example data so the code runs offline without downloading a huge dataset! On 10 random classes, random chance is 10% (`0.10`), and our classifier learns the training patterns up to `15.75%` in Stage d and `25.75%` in Stage e.



Output Graph from Cell [7]: Training accuracy rises from 10.13% to 15.75% over 10 epochs while training only the top classifier layers.

Block 7 — Cell [8] & Cell [9] (Stage e): Fine-Tuning by Unfreezing the Last 20 Layers

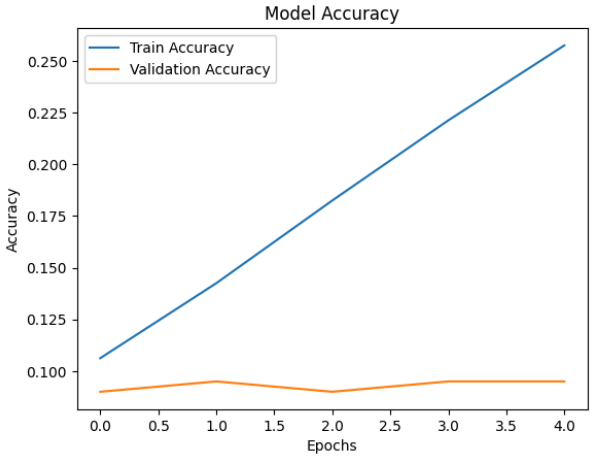
```
# Unfreeze some of the last convolutional layers for fine-tuning
for layer in base_model.layers[-20:]:  # Unfreeze last 20 layers; adjust as needed
    layer.trainable = True

# Recompile the model with a lower learning rate for fine-tuning
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Continue training with fine-tuning
fine_tune_history = model.fit(X_train, y_train, epochs=5, batch_size=32,
validation_split=0.2)
```

```
# Cell [9]: Plot training history (optional)
plt.plot(fine_tune_history.history['accuracy'], label='Train Accuracy')
plt.plot(fine_tune_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

- **How Fine-Tuning Works (Explain these 3 points in Viva!):**
  - **1.** **`for layer in base_model.layers[-20:]: layer.trainable = True`:** The slice `[-20:]` selects the **last 20 layers** of MobileNetV2 and unfreezes them (`trainable = True`), while keeping the first 134 layers frozen. We only unfreeze the *last* layers because early layers detect basic universal edges/colors, while the last layers detect high-level object features specific to our task.
  - **2. Why must we call `model.compile(...)` again?** In Keras, any change to `layer.trainable` only takes effect *after* you recompile the model!
  - **3. Why use a tiny learning rate `learning_rate=1e-5` (`0.00001`)?** Default Adam uses `0.001` (`1e-3`). During fine-tuning, we use a **100× smaller learning rate (`1e-5`)** so we gently nudge the pre-trained weights in the unfrozen layers without destroying their pre-trained knowledge!



Output Graph from Cell [9]: After unfreezing the last 20 layers of MobileNetV2 and fine-tuning at learning_rate=1e-5, training accuracy climbs from 10.63% to 25.75% in 5 epochs.

---

3. Top 6 External Viva Oral Questions & Ready-to-Speak Answers

Examiner Question	Simple, Confident Answer to Speak in Viva
1. What is Transfer Learning and why do we use it?	"Transfer Learning is a technique where we take a neural network already trained on a massive dataset (like MobileNetV2 trained on 1.4 million ImageNet photos) and reuse its learned feature-extraction layers for our own task. It saves days of training time and achieves high accuracy even with a small dataset."
2. What is the difference between Feature Extraction (Stage b–d) and Fine-Tuning (Stage e)?	"In **Feature Extraction**, we freeze the entire pre-trained base model (`trainable = False`) and train *only* the new classifier layers on top. In **Fine-Tuning**, after the top classifier has stabilized, we unfreeze the last few convolutional layers (`layers[-20:]`) and train them together using a very low learning rate (`1e-5`)."
3. Why did we set `include_top=False` when loading MobileNetV2?	"Because the original top layer of MobileNetV2 is a 1,000-neuron Dense layer built specifically for ImageNet's 1,000 classes. Setting `include_top=False` removes that final layer so we can attach our own custom `Dense(10)` classifier."
4. Why did we use `GlobalAveragePooling2D()` instead of `Flatten()`?	"If we used `Flatten()` on `(7, 7, 1280)`, it would create `7 × 7 × 1280 = 62,720` numbers, leading to over 16 million weights in the next Dense layer and severe overfitting! `GlobalAveragePooling2D()` averages each `7×7` map into 1 number, giving just `1,280` numbers and reducing parameters by 98%."
5. Why did we use a very low learning rate (`1e-5 = 0.00001`) during Fine-Tuning?	"Because the unfrozen layers of MobileNetV2 already contain well-trained weights. If we used a large learning rate, big gradient updates would wreck those pre-trained weights. A tiny learning rate (`1e-5`) allows gentle, incremental adjustments."
6. What makes MobileNetV2 special compared to older CNNs like VGG16?	"MobileNetV2 uses **Depthwise Separable Convolutions** and **Inverted Residual blocks**, which drastically reduce the number of parameters (~2.25M vs 138M in VGG16) so it can run fast even on mobile phones and CPUs."